

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: K.Rama

Roll No.: A24126552087

Section: CSM-B

Date: 22-09-2026

1. TITLE

Node.js File Management Application using the fs and readline Modules

2. AIM

To build a Node.js command-line application that creates a text file with user-supplied content, reads it back, appends additional content to it, and displays the file's content at each stage, using the built-in fs and readline modules.

3. OBJECTIVE

The objectives of this experiment are:

- To read user input from the terminal interactively using Node.js's readline module.
- To create, read, and append to files asynchronously using Node.js's fs module and callback functions.
- To chain multiple asynchronous operations correctly by nesting callbacks.
- To understand how package.json declares a Node.js project and its start script.

4. THEORY

Node.js's fs module provides both synchronous and asynchronous file operations; this experiment uses the asynchronous, callback-based versions — writeFile(), readFile(), and appendFile() — each of which takes a callback function that runs once the operation completes, receiving an error object (or null) and, for reads, the file's data. Because these calls are asynchronous, each step of the workflow (create, read, append, read again) is nested inside the callback of the previous step to guarantee it only runs after the earlier operation has actually finished; running them one after another at the top level would not work, since a later call could start before an earlier file operation had completed. The readline module's createInterface() connects to process.stdin and process.stdout to read lines typed by the user in the terminal; its question() method prints a prompt and asynchronously waits for the user to press Enter, passing the typed line to a callback — which is why each question() call is nested inside the previous one's callback. package.json declares the project's metadata and a "start" script (node app.js) so the application can be launched with "npm start".

5. ALGORITHM / PROCEDURE

1. Initialise a Node.js project with a package.json declaring app.js as the main file and a "start" script.
2. In app.js, require the built-in fs and readline modules, and create a readline interface connected to process.stdin and process.stdout.
3. Prompt the user with "Enter file name: " using rl.question(), and store the typed value as fileName.
4. Inside that callback, prompt with "Enter initial content: " and store the typed value as initialContent.
5. Build a file path files/{fileName}.txt and use fs.writeFile() to create the file with initialContent, logging an error or a success message in the callback.

6. Inside the write callback, use `fs.readFile()` to read the file back as UTF-8 text and print it under "Initial File Content:".
7. Inside that read callback, prompt with "Enter content to append: " and store the typed value as `appendContent`.
8. Use `fs.appendFile()` to add a newline followed by `appendContent` to the file, logging a success message once done.
9. Use `fs.readFile()` once more to read the final file content and print it under "Final File Content:".
10. Call `rl.close()` to end the readline session, and run the program with "node app.js" (or "npm start").

6. CODE EXPLANATION

`app.js` is a single sequential workflow built from nested asynchronous callbacks. A readline interface is created once at the top of the file and reused for every `question()`. The first two questions collect a file name and initial content; these are combined into a path of the form `files/{fileName}.txt`. `fs.writeFile()` then creates that file with the initial content, and once it succeeds, `fs.readFile()` immediately reads the same file back and logs it as the "Initial File Content", confirming the write worked. A third question then asks for content to append, and `fs.appendFile()` adds it to the file on a new line. Finally, the file is read one more time and logged as the "Final File Content", after which `rl.close()` ends the interactive session. Nesting each step inside the previous step's callback (rather than calling them one after another at the top level) is necessary because every `fs` and readline operation here is asynchronous — without the nesting, later steps could run before earlier ones had actually finished. `package.json`'s "main" field points to `app.js` and its "start" script allows the whole workflow to be launched with a single "npm start" command.

7. PROGRAM

`package.json`

```
{
  "name": "node-file-management",
  "version": "1.0.0",
  "description": "Node.js File Management PBL",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  }
}
```

`app.js`

```
const fs = require("fs");
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

rl.question("Enter file name: ", function(fileName) {

  rl.question("Enter initial content: ", function(initialContent) {
```

```

const filePath = `files/${fileName}.txt`;

fs.writeFile(filePath, initialContent, function(error) {

    if (error) {
        console.log("Error creating file:", error);
        rl.close();
        return;
    }

    console.log("\nFile created successfully.");

    fs.readFile(filePath, "utf8", function(error, data) {

        if (error) {
            console.log("Error reading file:", error);
            rl.close();
            return;
        }

        console.log("\nInitial File Content:");
        console.log(data);

        rl.question("\nEnter content to append: ", function(appendContent) {

            fs.appendFile(
                filePath,
                "\n" + appendContent,
                function(error) {

                    if (error) {
                        console.log("Error appending file:", error);
                        rl.close();
                        return;
                    }

                    console.log("\nContent appended successfully.");

                    fs.readFile(
                        filePath,
                        "utf8",
                        function(error, finalData) {

                            if (error) {
                                console.log(
                                    "Error reading final content:",
                                    error
                                );
                            }
                        }
                    );
                }
            );
        });
    });
});

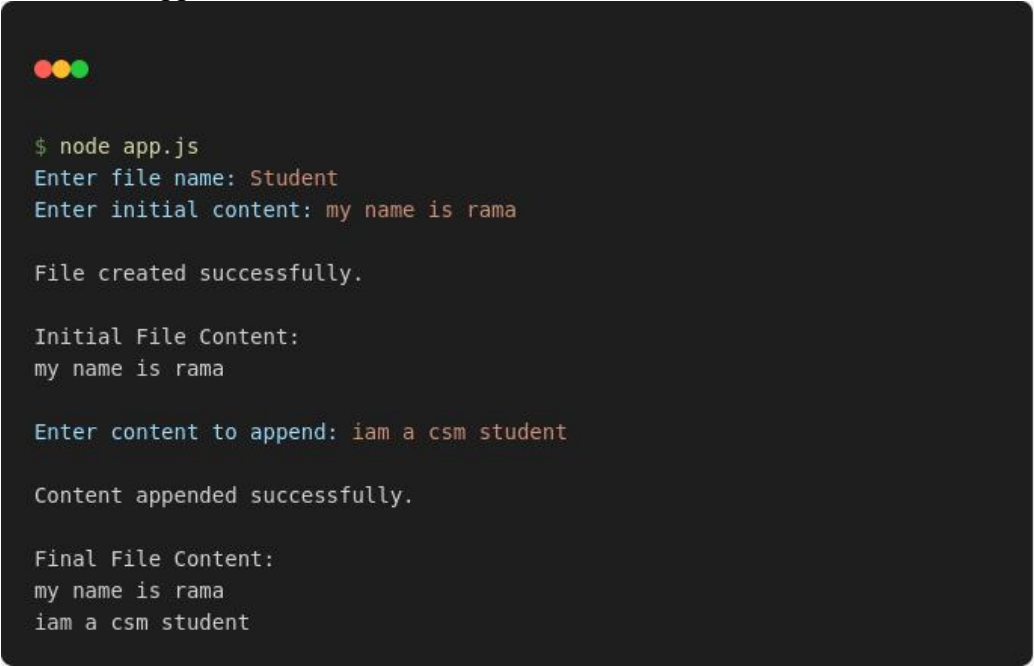
```

8. TEST CASES AND EXECUTION DATA

Anil Neerukonda Institute of Technology & Sciences (ANITS)

9. OUTPUT

Terminal session running "node app.js" with file name "Student", initial content "my name is rama", and appended content "iam a csm student":

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal shows the following text:

```
$ node app.js
Enter file name: Student
Enter initial content: my name is rama

File created successfully.

Initial File Content:
my name is rama

Enter content to append: iam a csm student

Content appended successfully.

Final File Content:
my name is rama
iam a csm student
```

Resulting files/Student.txt content after the run:

my name is rama
iam a csm student

10. RESULT

Thus, a Node.js command-line file management application was successfully built using the fs and readline modules. The application correctly created a file with user-supplied content, read it back, appended additional content, and displayed the final file content, all verified through an interactive terminal session.